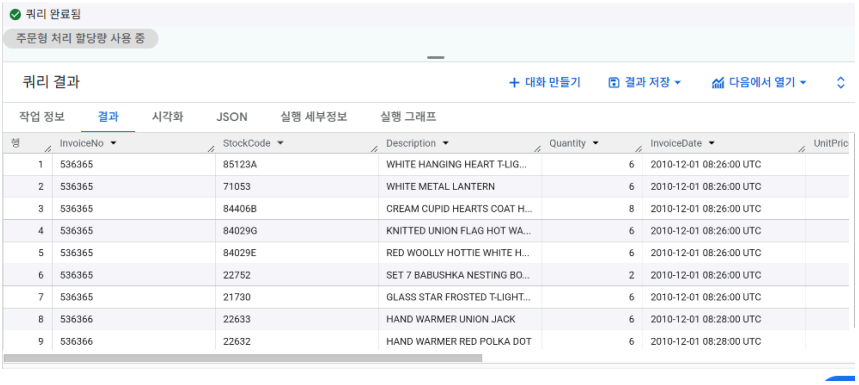
고객을 세그먼테이션하자 [프로젝트] - 권지훈

11-2. 데이터 불러오기

데이터 살펴보기

- **테이블에 있는 10개의 행만 출력하기**

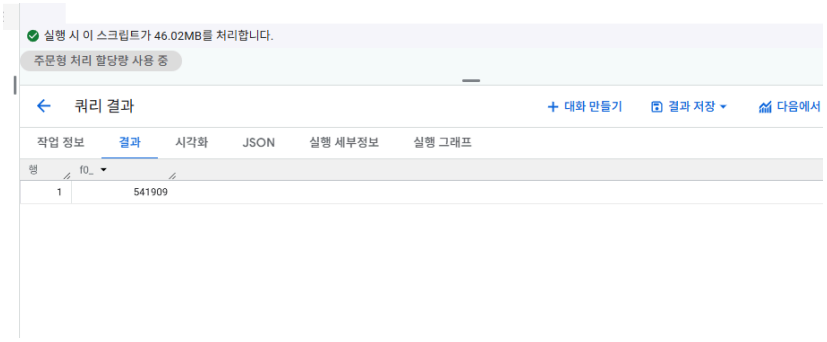
```
SELECT *
FROM modulabs_project.data
LIMIT 10;
```



행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPric
1	536365	85123A	WHITE HANGING HEART T-LIG...	6	2010-12-01 08:26:00 UTC	
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00 UTC	
3	536365	84406B	CREAM CUPID HEARTS COAT H...	8	2010-12-01 08:26:00 UTC	
4	536365	84029G	KNITTED UNION FLAG HOT WA...	6	2010-12-01 08:26:00 UTC	
5	536365	84029E	RED WOOLLY HOTTIE WHITE H...	6	2010-12-01 08:26:00 UTC	
6	536365	22752	SET 7 BABUSHKA NESTING BO...	2	2010-12-01 08:26:00 UTC	
7	536365	21730	GLASS STAR FROSTED T-LIGHT...	6	2010-12-01 08:26:00 UTC	
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:00 UTC	
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00 UTC	

- **전체 데이터는 몇 행으로 구성되어 있는지 확인하기**

```
SELECT COUNT(*)
FROM modulabs_project.data
```



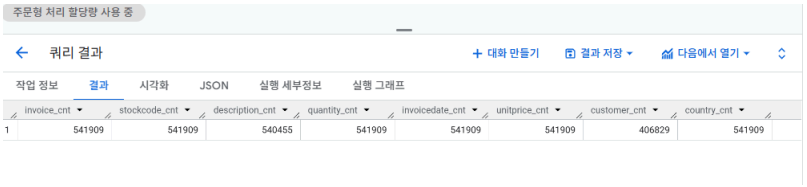
행	f0_
1	541909

데이터 수 세기

- **COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기**

```
SELECT
COUNT(InvoiceNo) AS invoice_cnt,
COUNT(StockCode) AS stockcode_cnt,
COUNT(Description) AS description_cnt,
COUNT(Quantity) AS quantity_cnt,
COUNT(InvoiceDate) AS invoicedate_cnt,
COUNT(UnitPrice) AS unitprice_cnt,
COUNT(CustomerID) AS customer_cnt,
```

```
COUNT(Country) AS country_cnt
FROM modulabs_project.data;
```



	invoice_cnt	stockcode_cnt	description_cnt	quantity_cnt	invoicedate_cnt	unitprice_cnt	customer_cnt	country_cnt
1	541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- **각 컬럼 별 누락된 값의 비율을 계산**
  - **각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기**

```
SELECT
'InvoiceNo' AS column_name,
ROUND(SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS null_ratio
FROM modulabs_project.data

UNION ALL

SELECT
'StockCode',
ROUND(SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
FROM modulabs_project.data

UNION ALL

SELECT
'Description',
ROUND(SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
FROM modulabs_project.data

UNION ALL

SELECT
'Quantity',
ROUND(SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
FROM modulabs_project.data

UNION ALL

SELECT
'InvoiceDate',
ROUND(SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
FROM modulabs_project.data

UNION ALL

SELECT
'UnitPrice',
ROUND(SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
FROM modulabs_project.data

UNION ALL

SELECT
'CustomerID',
```

```
ROUND(SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
FROM modulabs_project.data

UNION ALL

SELECT
'Country',
ROUND(SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
FROM modulabs_project.data;
```

행	column_name	null_ratio
1	InvoiceNo	0.0
2	StockCode	0.0
3	Description	0.27
4	Quantity	0.0
5	InvoiceDate	0.0
6	UnitPrice	0.0
7	CustomerID	24.93
8	Country	0.0

결측치 처리 전략

- `StockCode = '85123A'` 의 `Description` 을 추출하는 쿼리문을 작성하기

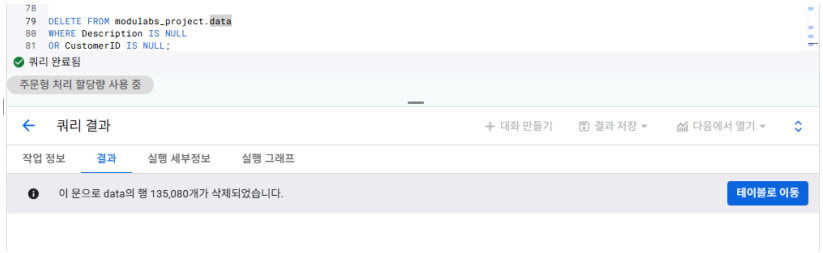
```
SELECT DISTINCT Description
FROM modulabs_project.data
WHERE StockCode = '85123A';
```

행	Description
1	WHITE HANGING HEART T-LIG...
2	?
3	wrongly marked carton 22804
4	CREAM HANGING HEART T-LIG...

결측치 처리

- **DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시**

```
DELETE FROM modulabs_project.data
WHERE Description IS NULL
OR CustomerID IS NULL;
```

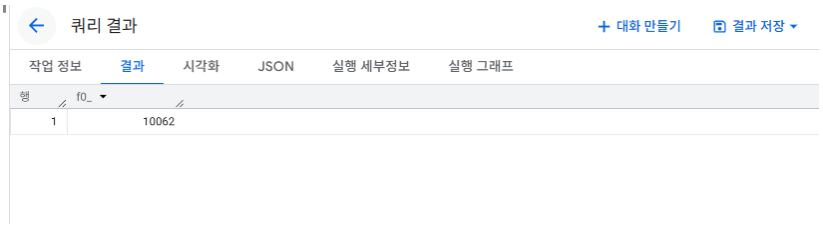


11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- **중복된 행의 수를 세어보기**
  - **8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기**

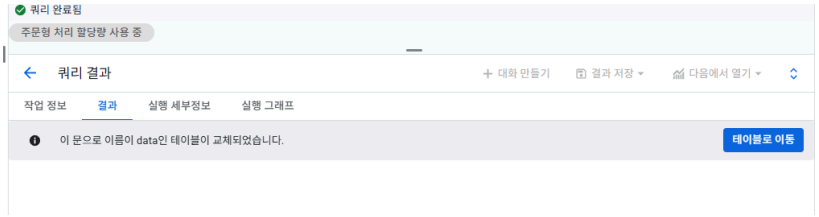
```
SELECT SUM(cnt)
FROM (
  SELECT COUNT(*) AS cnt
  FROM modulabs_project.data
  GROUP BY
  InvoiceNo,
  StockCode,
  Description,
  Quantity,
  InvoiceDate,
  UnitPrice,
  CustomerID,
  Country
  HAVING COUNT(*) > 1
);
```



중복값 처리

- **중복값을 제거하는 쿼리문 작성하기**
  - **CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트**

```
CREATE OR REPLACE TABLE modulabs_project.data AS
SELECT DISTINCT *
FROM modulabs_project.data;
```

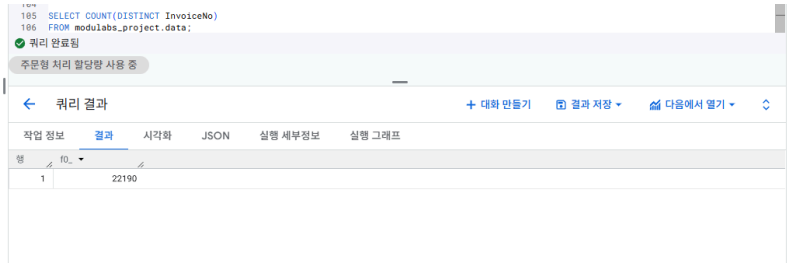


11-6. 데이터 전처리(3): 오류값 처리

`InvoiceNo` 살펴보기

- **고유(unique)한 `InvoiceNo` 의 개수를 출력하기**

```
SELECT COUNT(DISTINCT InvoiceNo)
FROM modulabs_project.data;
```



- **고유한 `InvoiceNo` 를 앞에서부터 100개를 출력하기**

```
SELECT DISTINCT InvoiceNo
FROM modulabs_project.data
LIMIT 100;
```

- **`InvoiceNo` 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)**

```
SELECT *
FROM modulabs_project.data
WHERE InvoiceNo LIKE 'C%'
LIMIT 100;
```

쿼리 결과

+ 대화 만들기 결과 저장 다음에서 열기

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPric
1	C541433	23166	MEDIUM CERAMIC TOP STORA...	-74215	2011-01-18 10:17:00 UTC	
2	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	
3	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	
4	C545330	M	Manual	-1	2011-03-01 15:49:00 UTC	
5	C547388	22701	PINK DOG BOWL	-6	2011-03-22 16:07:00 UTC	
6	C547388	84050	PINK HEART SHAPE EGG FRYIN...	-12	2011-03-22 16:07:00 UTC	
7	C547388	21914	BLUE HARMONICA IN BOX	-12	2011-03-22 16:07:00 UTC	
8	C547388	22413	METAL SIGN TAKE IT OR LEAVE...	-6	2011-03-22 16:07:00 UTC	
9	C547388	22784	LANTERN CREAM GAZEBO	-3	2011-03-22 16:07:00 UTC	

- **구매 건 상태가 `Canceled` 인 데이터의 비율(%) - 소수점 첫번째 자리까지**

```
SELECT ROUND(SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END) / COUNT(*) * 100, 1)
FROM modulabs_project.data;
```

쿼리 결과

+ 대화 만들기 결과 저장

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	f0_
1	2.2

`StockCode` 살펴보기

- **고유한 `StockCode` 의 개수를 출력하기**

```
SELECT COUNT(DISTINCT StockCode)
FROM modulabs_project.data;
```

주문형 처리 할당량 사용 중

쿼리 결과

+ 대화 만들기 결과 저장 다음에서

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	f0_
1	3684

- **어떤 제품이 가장 많이 판매되었는지 보기 위하여 `StockCode` 별 등장 빈도를 출력하기**
  - 상위 10개의 제품들을 출력하기

```
SELECT StockCode, COUNT(*) AS sell_cnt
FROM modulabs_project.data
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10;
```

쿼리 결과

+ 대화 만들기 결과

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	StockCode	sell_cnt
1	85123A	2065
2	22423	1894
3	85099B	1659
4	47566	1409
5	84879	1405
6	20725	1346
7	22720	1224
8	POST	1196
9	22197	1110
10	23203	1108

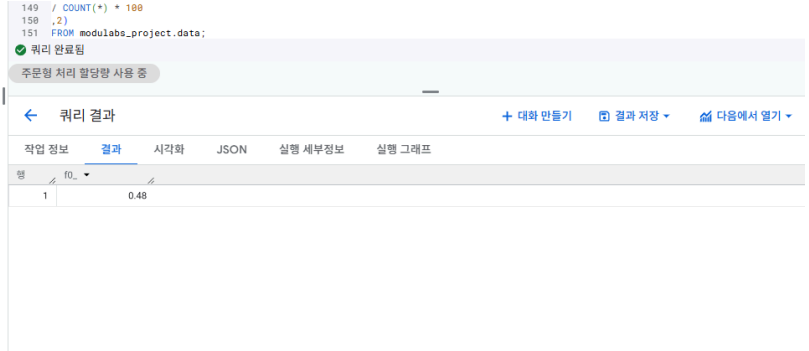
- `StockCode` **의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값**들에는 어떤 코드들이 들어가 있는지 출력하기

```
SELECT DISTINCT StockCode,
LENGTH(StockCode) - number_count AS word_cnt
FROM (
    SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
    FROM modulabs_project.data
)
WHERE number_count IN (0,1);
```

행	StockCode	word_cnt
1	POST	4
2	M	1
3	C2	1
4	D	1
5	BANK CHARGES	12
6	PADS	4
7	DOT	3
8	CRUK	4

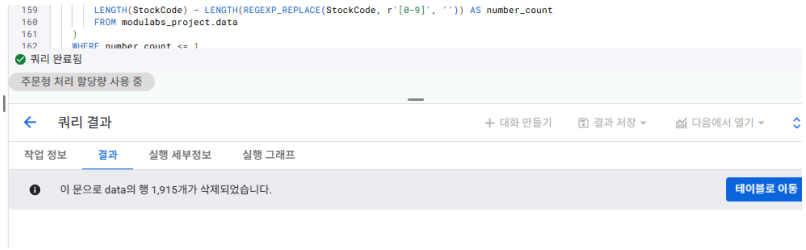
- `StockCode` **의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트**인지 구하기 (소수점 두 번째 자리까지)

```
SELECT
ROUND(
COUNTIF(
LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) <= 1
)
/ COUNT(*) * 100
,2)
FROM modulabs_project.data;
```



- **제품과 관련되지 않은 거래 기록을 제거하기**

```
DELETE FROM modulabs_project.data
WHERE StockCode IN (
    SELECT DISTINCT StockCode
    FROM (
        SELECT StockCode,
        LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
        FROM modulabs_project.data
    )
    WHERE number_count <= 1
);
```



`Description` 살펴보기

- **고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기**

```
SELECT Description, COUNT(*) AS description_cnt
FROM modulabs_project.data
GROUP BY Description
ORDER BY description_cnt DESC
LIMIT 30;
```

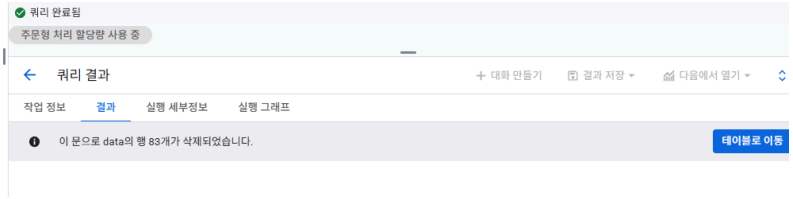
쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	Description	description_cnt
1	WHITE HANGING HEART T-LIG...	2058
2	REGENCY CAKESTAND 3 TIER	1894
3	JUMBO BAG RED RETROSPOT	1659
4	PARTY BUNTING	1409
5	ASSORTED COLOUR BIRD ORN...	1405
6	LUNCH BAG RED RETROSPOT	1345
7	SET OF 3 CAKE TINS PANTRY D...	1224
8	LUNCH BAG BLACK SKULL.	1099
9	PACK OF 72 RETROSPOT CAKE ...	1062

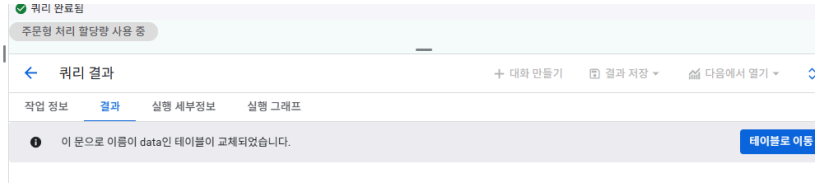
- **서비스 관련 정보를 포함하는 행들을 제거하기**

```
DELETE
FROM modulabs_project.data
WHERE Description IN ('Next Day Carriage', 'High Resolution Image');
```



- **대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기**

```
CREATE OR REPLACE TABLE modulabs_project.data AS
SELECT
  * EXCEPT (Description),
  UPPER(Description) AS Description
FROM modulabs_project.data;
```

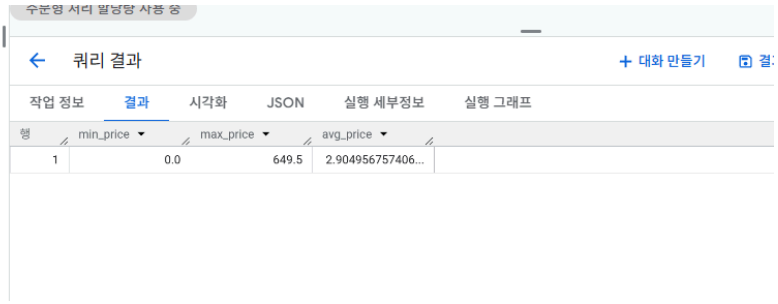


`UnitPrice` 살펴보기

- `UnitPrice` **의 최솟값, 최댓값, 평균을 구하기**

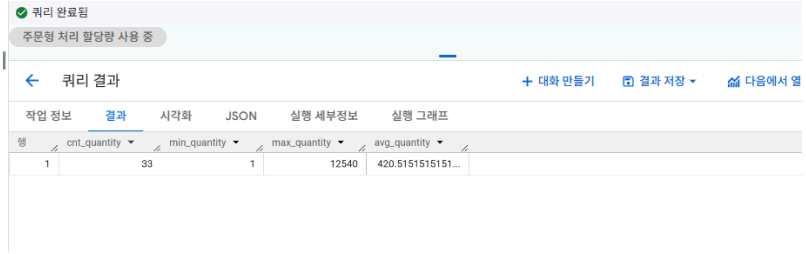
```
SELECT
MIN(UnitPrice) AS min_price,
MAX(UnitPrice) AS max_price,
```

```
AVG(UnitPrice) AS avg_price
FROM modulabs_project.data;
```



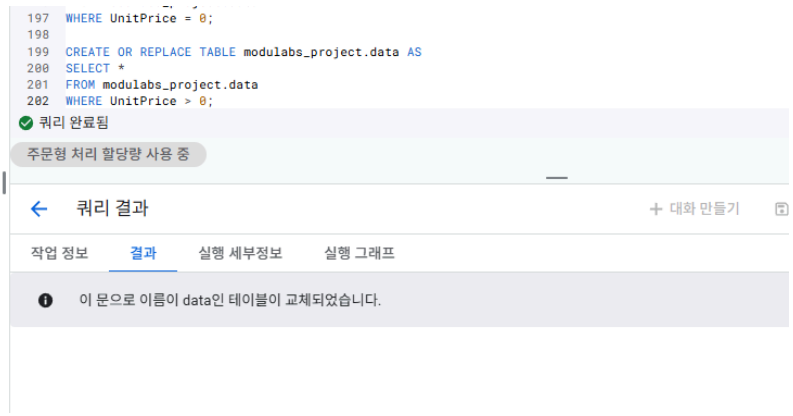
- **단가가 0원인 거래의 개수, 구매 수량(`Quantity`)의 최솟값, 최댓값, 평균 구하기**

```
SELECT
COUNT(Quantity) AS cnt_quantity,
MIN(Quantity) AS min_quantity,
MAX(Quantity) AS max_quantity,
AVG(Quantity) AS avg_quantity
FROM modulabs_project.data
WHERE UnitPrice = 0;
```



- **`UnitPrice = 0`를 제거하고 일관된 데이터셋을 유지하기**

```
CREATE OR REPLACE TABLE modulabs_project.data AS
SELECT *
FROM modulabs_project.data
WHERE UnitPrice > 0;
```

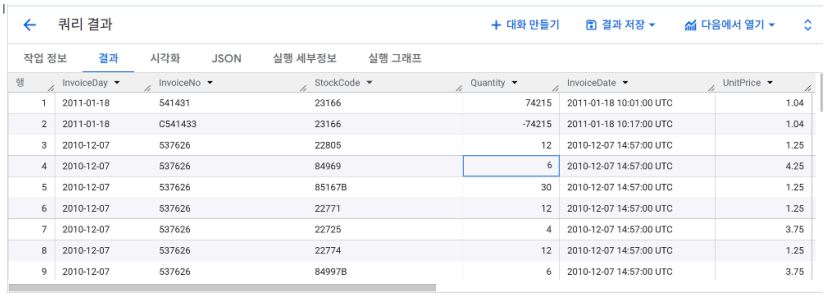


11-7. RFM 스코어

Recency

- `InvoiceDate` **컬럼을 연월일 자료형으로 변경하기**

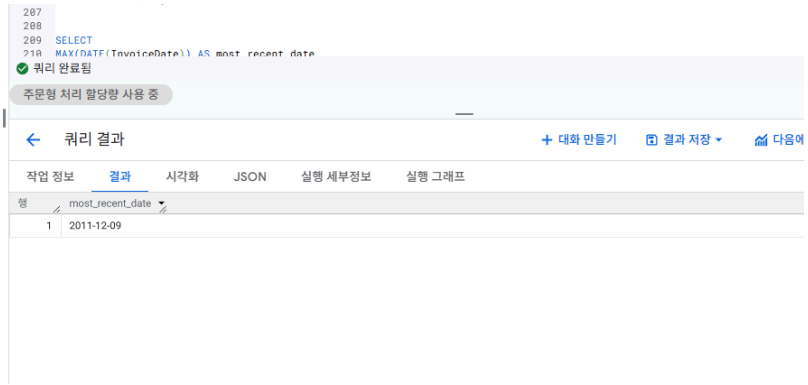
```
SELECT DATE(InvoiceDate) AS InvoiceDay, *
FROM modulabs_project.data;
```



행	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice
1	2011-01-18	541431	23166	74215	2011-01-18 10:01:00 UTC	1.04
2	2011-01-18	C541433	23166	-74215	2011-01-18 10:17:00 UTC	1.04
3	2010-12-07	537626	22805	12	2010-12-07 14:57:00 UTC	1.25
4	2010-12-07	537626	84969	6	2010-12-07 14:57:00 UTC	4.25
5	2010-12-07	537626	85167B	30	2010-12-07 14:57:00 UTC	1.25
6	2010-12-07	537626	22771	12	2010-12-07 14:57:00 UTC	1.25
7	2010-12-07	537626	22725	4	2010-12-07 14:57:00 UTC	3.75
8	2010-12-07	537626	22774	12	2010-12-07 14:57:00 UTC	1.25
9	2010-12-07	537626	84997B	6	2010-12-07 14:57:00 UTC	3.75

- **가장 최근 구매 일자를 MAX() 함수로 찾아보기**

```
SELECT
MAX(DATE(InvoiceDate)) AS most_recent_date
FROM modulabs_project.data;
```



행	most_recent_date
1	2011-12-09

- **유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기**

```
SELECT
CustomerID,
MAX(DATE(InvoiceDate)) AS InvoiceDay
FROM modulabs_project.data
GROUP BY CustomerID;
```

← 쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행

행	CustomerID	InvoiceDay
1	12346	2011-01-18
2	12347	2011-12-07
3	12348	2011-09-25
4	12349	2011-11-21
5	12350	2011-02-02
6	12352	2011-11-03
7	12353	2011-05-19
8	12354	2011-04-21
9	12355	2011-05-09

- **가장 최근 일자(`most_recent_date`)와 유저별 마지막 구매일(`InvoiceDay`)간의 차이를 계산하기**

```
SELECT
CustomerID,
EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
    SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
    FROM modulabs_project.data
    GROUP BY CustomerID
);
```

← 쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

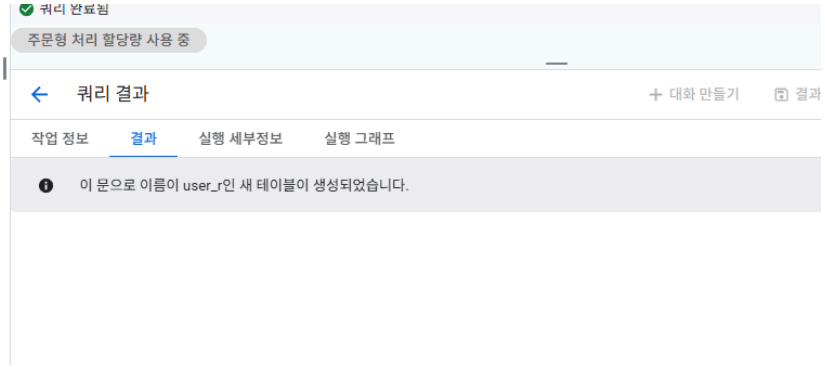
행	CustomerID	recency
1	12363	109
2	12397	35
3	12520	79
4	12614	277
5	12851	96
6	12854	78
7	12875	143
8	12952	5
9	13015	53

서비스 제공 및 서비스 품질 개선과 트래픽 분석을 위해 Google 쿠키를 사용합니다.

- **최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기**

```
CREATE OR REPLACE TABLE modulabs_project.user_r AS
SELECT
CustomerID,
EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
SELECT
CustomerID,
MAX(DATE(InvoiceDate)) AS InvoiceDay
FROM modulabs_project.data
```

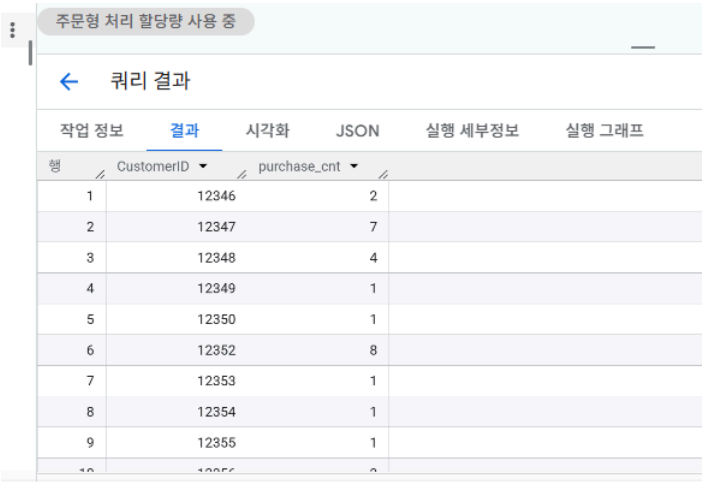
```
GROUP BY CustomerID
);
```



Frequency

- **고객마다 고유한 InvoiceNo의 수를 세어보기**

```
SELECT
CustomerID,
COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM modulabs_project.data
GROUP BY CustomerID;
```



행	CustomerID	purchase_cnt
1	12346	2
2	12347	7
3	12348	4
4	12349	1
5	12350	1
6	12352	8
7	12353	1
8	12354	1
9	12355	1

비스 제공 및 서비스 품질 개선과 트래픽 분석을 위해 Google 쿠키를 사용합니다.

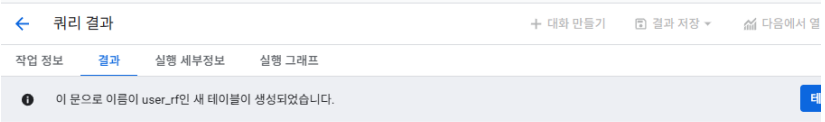
- **각 고객 별로 구매한 아이템의 총 수량 더하기**

```
SELECT
CustomerID,
SUM(Quantity) AS item_cnt
FROM modulabs_project.data
GROUP BY CustomerID;
```

행	CustomerID	item_cnt
1	12346	0
2	12347	2458
3	12348	2332
4	12349	630
5	12350	196
6	12352	463
7	12353	20
8	12354	530
9	12355	240

- **전체 거래 건수 계산와 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기**

```
CREATE OR REPLACE TABLE modulabs_project.user_rf AS

-- (1) 전체 거래 건수 계산
WITH purchase_cnt AS (
SELECT
CustomerID,
COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM modulabs_project.data
GROUP BY CustomerID
),

-- (2) 구매한 아이템 총 수량 계산
item_cnt AS (
SELECT
CustomerID,
SUM(Quantity) AS item_cnt
FROM modulabs_project.data
GROUP BY CustomerID
)

-- 기존 user_r에 (1)과 (2)를 통합
SELECT
pc.CustomerID,
pc.purchase_cnt,
ic.item_cnt,
ur.recency
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
ON pc.CustomerID = ic.CustomerID
JOIN modulabs_project.user_r AS ur
ON pc.CustomerID = ur.CustomerID;
```



Monetary

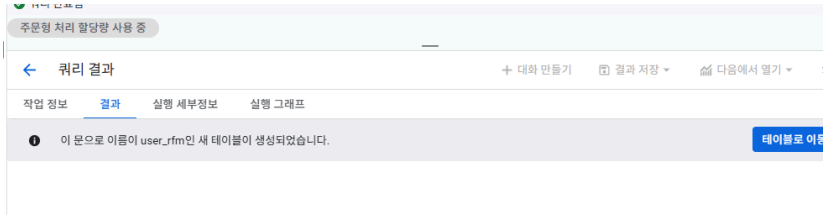
- **고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)**

```
SELECT
CustomerID,
ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
FROM modulabs_project.data
GROUP BY CustomerID;
```

행	CustomerID	user_total
1	12346	0.0
2	12347	4310.0
3	12348	1437.2
4	12349	1457.5
5	12350	294.4
6	12352	1265.4
7	12353	89.0
8	12354	1079.4
9	12355	459.4

- **고객별 평균 거래 금액 계산**
  - **고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인(LEFT JOIN) 한 후, 2) `purchase_cnt`로 나누어서 3) `user_rfm` 테이블로 저장하기**

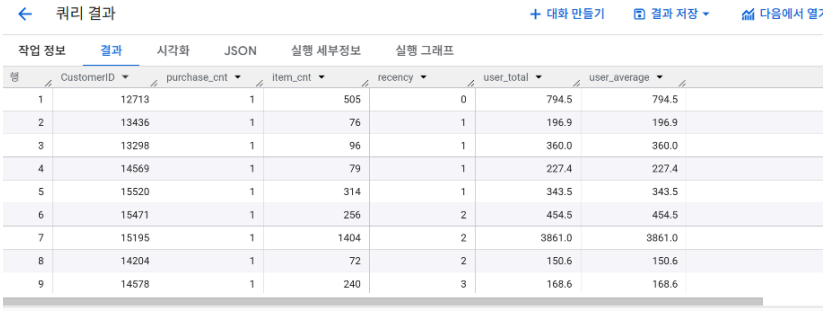
```
CREATE OR REPLACE TABLE modulabs_project.user_rfm AS
SELECT
rf.CustomerID AS CustomerID,
rf.purchase_cnt,
rf.item_cnt,
rf.recency,
ut.user_total,
ROUND(ut.user_total / rf.purchase_cnt, 1) AS user_average
FROM modulabs_project.user_rf rf
LEFT JOIN (
    -- 고객별 총 지출액
    SELECT
    CustomerID,
    ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
    FROM modulabs_project.data
    GROUP BY CustomerID
) ut
ON rf.CustomerID = ut.CustomerID;
```



RFM 통합 테이블 출력하기

- **최종 user_rfm 테이블을 출력하기**

```
SELECT *
FROM modulabs_project.user_rfm;
```



행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.5	794.5
2	13436	1	76	1	196.9	196.9
3	13298	1	96	1	360.0	360.0
4	14569	1	79	1	227.4	227.4
5	15520	1	314	1	343.5	343.5
6	15471	1	256	2	454.5	454.5
7	15195	1	1404	2	3861.0	3861.0
8	14204	1	72	2	150.6	150.6
9	14578	1	240	3	168.6	168.6

11-8. 추가 Feature 추출

1. 구매하는 제품의 다양성

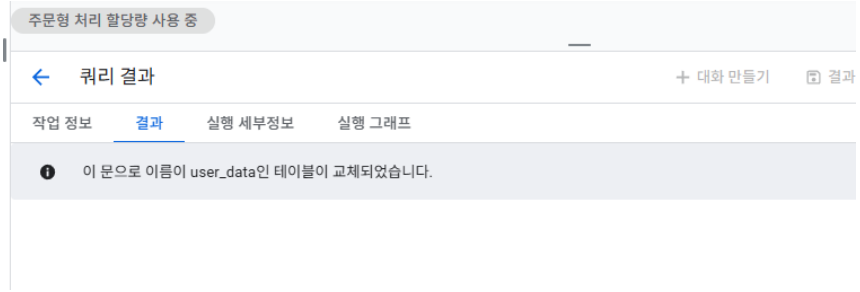
- **1) 고객 별로 구매한 상품들의 고유한 수를 계산하기**
  2) `user_rfm` 테이블과 결과를 합치기
  3) `user_data` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE modulabs_project.user_data AS
WITH unique_products AS (
SELECT
CustomerID,
COUNT(DISTINCT StockCode) AS unique_products
FROM modulabs_project.data
GROUP BY CustomerID
)

SELECT
ur.,
up. EXCEPT (CustomerID)
FROM modulabs_project.user_rfm AS ur
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID;
```

2. 평균 구매 주기

- **고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)**
  - **균 구매 소요 일수를 계산하고, 그 결과를 `user_data` 에 통합**

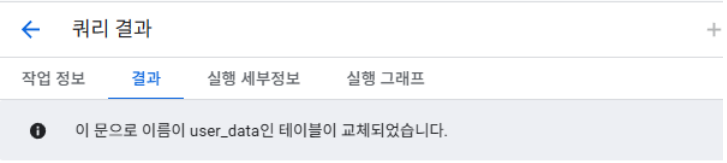
```
CREATE OR REPLACE TABLE modulabs_project.user_data AS

WITH purchase_intervals AS (

-- (2) 고객 별 구매와 구매 사이 평균 소요 일수
SELECT
CustomerID,
CASE
WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0
ELSE ROUND(AVG(interval_), 2)
END AS avg_purchase_interval

FROM (

-- (1) 구매와 구매 사이 소요된 일수
SELECT
CustomerID,
DATE_DIFF(
DATE(InvoiceDate),
LAG(DATE(InvoiceDate)) OVER (
PARTITION BY CustomerID
ORDER BY InvoiceDate
),
DAY
) AS interval_

FROM modulabs_project.data
WHERE CustomerID IS NOT NULL

)

GROUP BY CustomerID
)

SELECT
u.*,
pi.* EXCEPT(CustomerID)

FROM modulabs_project.user_data AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;
```



3. 구매 취소 경향성

- **고객의 취소 패턴 파악하기**
  1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수
  2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율
  - **취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data` 에 통합하기**
    (취소 비율은 소수점 두번째 자리)

```
CREATE OR REPLACE TABLE modulabs_project.user_data AS

WITH TransactionInfo AS (

SELECT
CustomerID,
COUNT(DISTINCT InvoiceNo) AS total_transactions,
COUNTIF(InvoiceNo LIKE 'C%') AS cancel_frequency
FROM modulabs_project.data
GROUP BY CustomerID

)

SELECT
u.*,
t.* EXCEPT(CustomerID),
ROUND(cancel_frequency / total_transactions, 2) AS cancel_rate

FROM modulabs_project.user_data AS u
LEFT JOIN TransactionInfo AS t
ON u.CustomerID = t.CustomerID;
```



- **다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 `user_data` 를 출력하기**

```
SELECT *
FROM modulabs_project.user_data;
```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	avg_purchase_int...
1	16252	1	-158	366	-295.1	-295.1	21	0.
2	15823	1	-282	337	-811.9	-811.9	15	0.
3	12536	2	2672	7	4151.3	2075.7	259	0.1
4	12454	2	0	53	0.0	0.0	15	0.
5	18072	2	48	155	12.2	6.1	19	0.9
6	14557	2	0	64	0.0	0.0	16	0.6
7	12607	2	0	58	0.0	0.0	101	0.0
8	14205	3	100	16	228.5	76.2	34	9.4
9	12967	4	243	3	1194.8	298.7	21	11.

회고

Keep : 효율적으로 쿼리를 작성하는것도 중요하지만 실제상황이라고 생각하고 어떤식으로 단계적으로 정리하면서 어떤 흐름으로 만들어야 하는지 생각하면서 진행했습니다.

Problem : 거의 모든 부분이 문제였지만 추가 피처를 만드는 부분에서 많이해맨부분이 많고 frm분석을 할때에도 제대로 된건지 확신이 들지않았습니다.

Try : 지금 거의 한가지의 방법으로 쭉 해왔는데 다른 대체 방법으로도 진행해보고 제가 흥미있어하는 데이터로 더 연습해보려고합니다.